Learned Tobler: Architecture and Setup

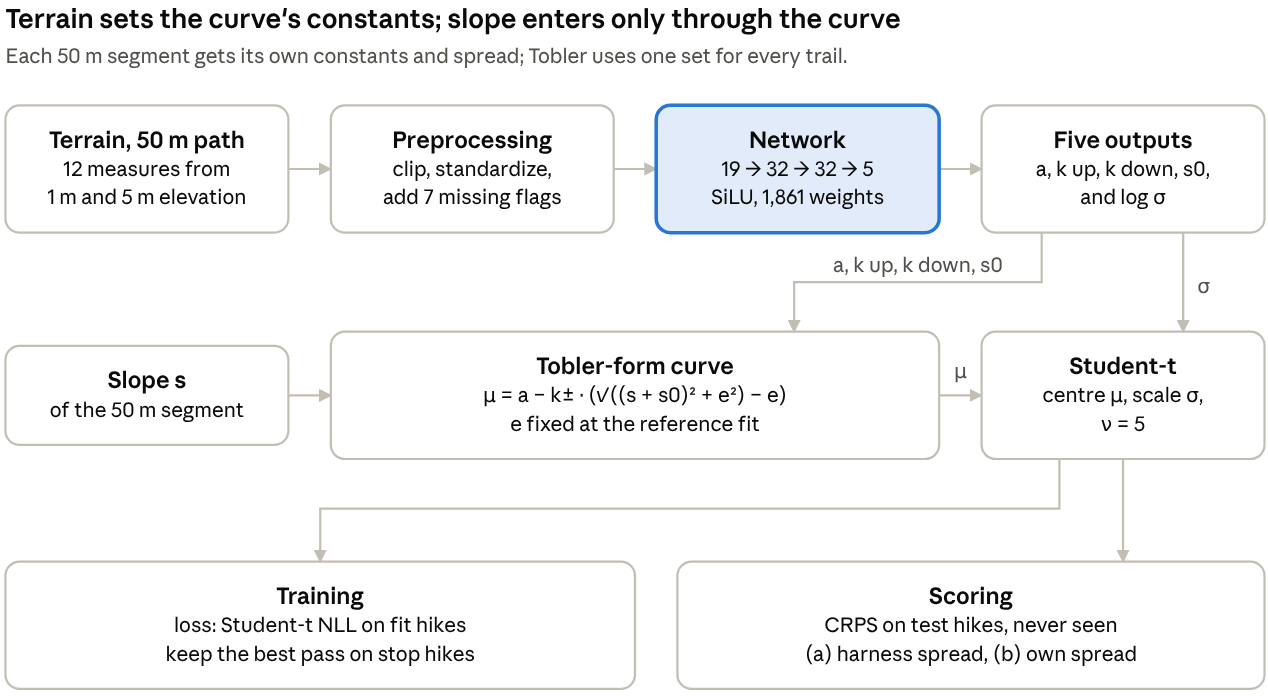
Oct 29, 2026 · @Brian Zhou

In short

Learned Tobler keeps Tobler's curve of speed against slope, but a small neural network sets the curve's constants for each 50 m trail segment from that segment's terrain. The network sees only terrain; slope enters through the curve itself.

- **What it predicts, per segment:** a peak speed, how fast speed falls going uphill and going downhill, where the peak sits, and how uncertain the prediction is.
- **What it is compared with:** boosted trees given the same terrain inputs, the same folds and the same scoring.
- **What it does not use:** map tags, vegetation, satellite embeddings, coordinates or which region a segment is in.

Architecture



learned Tobler · inputs, network, curve, likelihood

Only the blue box is learned. The curve's shape and the likelihood are fixed, so every prediction is still a Tobler curve, with its own constants for each segment.

The model

For a segment with slope s (rise over run in the walking direction) and terrain features x, the predicted log speed is Tobler's form with a rounded top and separate uphill and downhill decay:

$$\mu = a(x) - k_{\pm}(x)\left(\sqrt{\left(s + s_0(x)\right)^2 + e^2} - e\right), \qquad k_{\pm} = \begin{cases} k_{\mathrm{up}}(x) & s + s_0(x) \ge 0 \\ k_{\mathrm{down}}(x) & s + s_0(x) < 0 \end{cases}$$

The observed log speed y = ln(speed in km/h) is modelled as Student-t with centre μ, scale σ(x) and ν = 5. With e = 0, k up = k down = 3.5, a = ln 6 and s0 = 0.05, the formula is exactly Tobler's 1993 function.

Output	Meaning	Constraint	Starting value
a	log of the peak speed	none	reference fit, ≈ 1.43 (4.2 km/h)
k up	how fast speed falls going uphill	softplus, > 0	reference fit, ≈ 2.6
k down	how fast speed falls going downhill	softplus, > 0	reference fit, ≈ 2.4
s0	where the peak sits (the peak is at slope −s0)	none	reference fit, ≈ 0.037
log σ	spread of the Student-t	exp, σ > 0	log of the SD of the calibration residuals

The rounding e is not learned; it stays at the reference fit, ≈ 0.1. The **reference fit** is the same curve with one set of constants for every segment, fitted per fold to the training hikes by robust least squares (soft-L1 loss, scale 0.3). It is also the slope-only baseline that every skill below is measured against.

Inputs

The network sees 12 terrain measures of the segment's own path, sampled every 1 m along the matched trail line from two elevation models: the 5 m USGS grid (all 17 regions) and the 1 m lidar grid (the 16 US regions).

Measure	5 m version	1 m version	What it captures
Steepest pitch	pitch_max_5m	pitch_max	the steepest 5 m stretch, |Δz| / 5 m
Steppiness	pitch_sd_5m	pitch_sd	SD of the 5 m pitches along the segment
Relief along the path	relief_sd_5m	relief_sd	SD of the profile around its 11 m moving average: rock steps, roots, water bars
Hidden climbing	hidden_grade_5m	hidden_grade	extra Tobler time when the 50 m is walked as 5 m pieces rather than at its average grade
Side slope	cross_slope_5m	cross_slope	mean slope across the path
Roughness	—	rough_mad	mean |ground − ground smoothed over 2 m| on the path
Roughness, STRIDE's definition	—	rough_stride	the same with STRIDE's 2 m circular mean filter

Preprocessing is fitted on each fold's training rows only:

1. **Clip** each measure to the training rows' 0.1st–99.9th percentiles. A few elevation glitches at cliff edges and stairways sit hundreds of SDs out, and a network would extrapolate them.
2. **Standardize** with the training mean and SD.
3. **Missing values:** the North Shore (Canada) has no 1 m lidar, so its seven 1 m measures are missing (5.1% of segments). They are set to 0, the training mean, and a 0/1 missing flag per measure is added whenever the training rows have gaps.

The network therefore has 19 inputs (12 measures and 7 flags), or 12 in the fold that holds out the North Shore. With two hidden layers of 32 units that makes 1,861 trainable weights (1,637 with 12 inputs).

Training

In every fold the training hikes are split by hike into three roles, about 70 / 15 / 15%: **fit** (learned from), **stop** (decides when training ends) and **cal** (measures the residual spread used in scoring). These are the evaluation harness's own assignments, so the trees use exactly the same rows.

- **Loss:** the mean Student-t negative log-likelihood of y, with $\nu = 5$ fixed. The heavy tails keep GPS glitches and missed stops from dominating.
- **Network:** fully connected, inputs → 32 → 32 → 5, SiLU activations, PyTorch's default initialization for the hidden layers.
- **Start:** the output layer's weights are zero and its biases are the reference constants, so the untrained model is the reference curve. Its predictions match the reference to within 3×10^{-7} in every fold.
- **Optimizer:** Adam, learning rate 10^{-3}, PyTorch's defaults otherwise, no weight decay; batches of 2,048 fit segments, reshuffled every pass.
- **Early stopping:** after each pass, the loss over all stop segments. The best weights are kept, and training ends after 5 passes without an improvement of at least 10^{-5}, or at 60 passes. On the new-hikes folds this took 13–23 passes.
- **Seeds:** 0, 1 and 2, each setting the starting weights and the batch order. Scores average the three seeds segment by segment.
- **Hardware:** a laptop CPU in float32. All five setups with three seeds, plus the trees, take about 6 minutes.

Evaluation

There are five test setups. In each fold the test segments are never used for fitting, stopping or calibration.

Setup	Held out in each fold	Folds	Intervals resample
New hikes	a fifth of the hikes	5	hikes

Setup	Held out in each fold	Folds	Intervals resample
New hikers	a fifth of the uploaders, with all their hikes	5	uploaders
New 10 km squares	a fifth of the 10 km map squares	5	hikes
Unseen regions	one whole region	17	regions
Unseen regions and hikers	one region, plus every hiker who walked there	17	regions

All scores are on log speed:

- **CRPS** (continuous ranked probability score) grades the whole predicted distribution, not only its centre; lower is better.
- **Skill** = 100 × (1 − Σ CRPS of the model / Σ CRPS of the reference) over all test segments of a setup, where the reference is the refitted slope curve.
- **(a) Harness spread:** the model's μ plus the 100 quantiles of its residuals on the cal hikes, the same rule every model in the paper's Table I uses. This compares the centres on equal terms.
- **(b) Own spread:** the model's own Student-t(μ, σ, 5), scored in closed form.
- **Calibration:** the share of test segments inside the model's own central 90% range.
- **Intervals:** 95% bootstrap over 2,000 draws of the setup's units; differences between models are paired within each draw.

The trees baseline is the harness's terrain model (gbm_rs) refitted on the same folds: scikit-learn's HistGradientBoostingRegressor on slope plus the 12 raw terrain measures, with missing values handled by the trees. Settings: learning rate 0.05, up to 1,000 trees of 31 leaves, at least 200 segments per leaf, L2 penalty 1.0, early stopping on the stop hikes after 30 rounds without improvement, and the spread from the cal residual quantiles. It reproduces the paper's values to within 0.02 points.

Results from this setup

Scored the harness's way, learned Tobler ties the trees in every setup; with its own spread it is ahead on new hikes, new 10 km squares and unseen regions.

Setup	Trees	Learned Tobler, harness spread (a)	Learned Tobler, own spread (b)	Own 90% range holds
New hikes	+3.22%	+3.19%	+3.77%	89.8%
New hikers	+3.19%	+3.07%	+3.36%	89.9%
New 10 km squares	+2.83%	+2.90%	+3.50%	89.7%
Unseen regions	+2.79%	+3.01%	+3.37%	89.4%
Unseen regions and hikers	+2.42%	+2.55%	+3.05%	89.4%

Skill is the improvement in CRPS over the refitted slope curve. Paired with the trees, (a) differs by −0.12 to +0.22 points with every 95% interval spanning zero; (b) leads by +0.55 to +0.66 points on new hikes, new squares and unseen regions, with intervals above zero.

Reproduce

Everything above runs from travel-speed-gate1/exercises/week10_learned_tobler/ in the private repo Brian24NX/GeoAI:

- `learned_tobler.py`: the model, loss, training and both scorings for one setup.
- `run_all.py`: all five setups with seeds 0–2, and the trees on the same folds.
- `gate2_summary.py`: skills, bootstrap intervals and paired differences.
- `plot_constants.py`: the learned constants against steppiness.

```
cd travel-speed-gate1/exercises/week10_learned_tobler
../../../.venv/bin/python run_all.py        # 5 setups x seeds 0-2, plus the
trees: ~6 min
../../../.venv/bin/python gate2_summary.py  # ->
results/t3_learned_tobler/gate2_summary.csv
```

- **Versions:** Python 3.12.12, PyTorch 2.14.1, scikit-learn 1.9.1, NumPy 2.5.3, SciPy 1.18.1, pandas 3.0.6.
- **Data:** the de-identified modelling table `cache/exercise/t3_table.parquet`, with the harness's fold and role assignments; not in git. It holds 289,679 scored segments from 1,992 hikes by 881 uploaders in 17 regions.

Code and this description were developed with AI assistance (Claude).